LENGUAJE DE PROGRAMACION

Es un conjunto de reglas sintácticas, semánticas y símbolos formales que permite a un ser humano escribir instrucciones estructuradas para que una computadora las interprete, procese y ejecute.

ELEMENTOS ESENCIALES DE UN LENGUAJE DE PROGRAMACIÓN

- **SINTAXIS** -. Es el conjunto de reglas gramaticales que dictan como deben escribirse las palabras y símbolos estructurados para que el código sea válido.
- **SEMÁNTICA** -. Es el significado o la acción lógica que realiza cada instrucción escrita
- **VOCABULARIO / PALABRAS RESERVADAS** :. Palabras clave con significado especial para el lenguaje que no pueden usarse para otros fines. Ejm: print, cout, while, return, etc.
- **PROGRAMACIÓN** :. Es el proceso de diseñar, escribir, programar y mantener código fuente para dar instrucciones a una computadora permitiendole ejecutar tareas específicas o resolver problemas de forma automatizada.
- **ALGORITMO** -. Es una secuencia finita, ordenada y lógica de pasos o instrucciones precisas que deben seguirse para realizar una tarea o resolver un problema concreto.
- **LENGUAJE DE ALTO NIVEL** :. Es un lenguaje diseñado para ser leído y escrito facilmente por los seres humanos. Utiliza palabras en inglés y estructuras lógicas cercanas al lenguaje nativo, independizando al programador de la arquitectura interna del hardware. Ejm: C++, Python, Java Script, etc.
- **LENGUAJE DE BAJO NIVEL** :. Es un lenguaje que opera con mínima o nula abstracción del hardware diseñado para comunicarse e interactuar directamente con la memoria y el procesador de la computadora.

andes

Escaneado con CamScanner

Entorno desarrollado integrado (IDE)

Es una aplicación de software que reúne todas las herramientas necesarias para que un desarrollador escriba y pruebe código en un solo lugar

COMPONENTES PRINCIPALES DE UN IDE :

- EDITOR DE CÓDIGO FUENTE : Incluye resaltado de sintaxis, autocompletado y detección de errores en tiempo real.
- COMPILADOR / INTEPRETADOR INTEGRADO : Permite ejecutar el programa con un solo click.
- DEPURADOR (DEBUGGER) : Herramienta para inspeccionar la memoria, avanzar línea por línea y rastrear errores.
- GESTOR DE PROYECTOS : Organizo múltiples archivos, bibliotecas y conefiguraciones del programa.

VARIABLE - Una variable es un espacio reservado en la memoria RAM del ordenador que se utiliza para almacenar un valor que puede cambiar a lo largo de la ejecución del programa tiene un nombre (identificador) y un tipo de dato asociado. ejm. convertir grados °C a °F

CONSTANTE - Una constante es un valor almacenado en memoria que no puede cambiar ni ser modificado una vez que ha sido definido durante la ejecución del programa. Se utilizan para evitar errores y representar valores fijos del dominio.
ejm.

andes

Escaneado con CamScanner

Tipos de datos

Categoría	Tipo de dato	Tamaño habitual	Rango (aproximado)
Enteros	int	4 bytes	-2147483648 a 2147483647
	short	2 bytes	-32768 a 32767
	long	4 u 8 bytes	-2147483648 a 2147483647
	long long	8 bytes	-9223372036854775808 a 9223372036854775807
Punto flotante	float	4 bytes	$\pm 3.4 \times 10^{-38}$ a $\pm 3,4 \times 10^{38}$
	double	8 bytes	$\pm 1.7 \times 10^{308}$ a $\pm 1,7 \times 10^{308}$
Caracteres	Char	1 byte	-128 a 127
	Wchar_t	2 o 4 bytes	0 a 65535 / 0 a 4294967295
Lógicos	bool	1 byte (típico)	0 o 1

DFD (Diagrama de Flujo de Datos)

Es una representación gráfica que muestra cómo fluye la información a través de un sistema o proceso.

Símbolo	Nombre	Función
Cuadro / rectángulo	Entidad externa	Representa la fuente u origen de los datos.
Círculo / óvalo	Proceso	Operación o función que transforma los datos.
Líneas horizontales / rectángulo abierto	Almacén de datos	Lugar donde se guardan los datos temporal o permanentemente.
Flecha	Flujo de datos	Muestra la dirección en la que viaja la información entre los elementos.

Escaneado con CamScanner

PSeInt

Es una herramienta educativa de software libre diseñada para ayudar a estudiantes a dar sus primeros pasos en la programación y el desarrollo de la lógica algorítmica
Utiliza pseudocódigo en español, lo que permite centrarse en los conceptos fundamentales de la programación (variables, condicionales, bucles y funciones) sin la complicación de la sintaxis estricta de un lenguaje real (como C++, Java o Python)

- Conversión de tiempo (segundos a minutos/horas)
  Leer un valor en segundos e imprimir a cuantos minutos y horas equivale.
- Calculo del IGV de un producto
  Solicita el precio base de un producto y calcula el monto del IGV (18%) junto con el precio total final.
- Área y perímetro de un rectángulo
  Pide la longitud de la base y la altura de un rectángulo para imprimir su área y perímetro.

- Operadores Aritméticos : +, -, *, /, %, ++, --.
- Operadores Relacionales : ==, !=, >, <, >=, <=.
- Operadores Lógicos : &&, ||, !.

ESTRUCTURAS CONDICIONALES

if - else

Las estructuras if - else son sentencias de control condicionales que permiten tomar decisiones en el código ejecutando bloques de instrucción únicamente cuando se cumple una condición.

- Estructura if (condicional simple) Evalúa una condición: Si la condición resulta verdadera (true), se ejecuta el código dentro del bloque. Si es falsa (false), el programa omite ese bloque y continúa con la siguiente instrucción
- Estructura if - else (condicional doble) Permite un camino alternativo. Si la condición resulta verdadera (true) se ejecuta el prime bloque. Si es falso (false) se ejecuta obligatoriamente el bloque dentro el else.

Escaneado con CamScanner

Ejemplo: par o impar

```
if (numero %2== = 0)
      // par
else
      // impar
```

Otro caso con if/else if : Clasificar un número como positivo, negativo o nulo

switch-case

En C++ switch/case es una estructura de control de selección múltiple que evalúa el valor de una variable y ejecuta un bloque de código según la coincidencia con una lista de casos enteros o caracteres.

Ejemplo:

```
Switch (variable) {
    case var1:
        // ejecuta un código
        break;
    Case var2:
        // ejecuta un código
        break;
    default:
        // Código : no coincide con ninguno previo caso
        break;
}
```

Escaneado con CamScanner

Estructuras repetitivas

También llamada bucle, ciclo o iteración. Es un bloque de código que permite ejecutar una o varias instrucciones de forma recurrente mientras se cumpla una determinada condición lógica.
Su propósito principal es automatizar tareas repetitivas, evitar la duplicidad de código y procesar colecciones de datos (como listas o arreglos) de manera eficiente.

Ejemplo:

```
for(inicio ; final ; incremento)
for(i=1 ; i<=10 ; i++)
```

- **Iteración:** Una iteración es cada una de las repeticiones o ejecuciones individuales de un bucle. Si un bucle se ejecuta 5 veces, se dice que realiza 5 iteraciones.

- **Contador:** Es una variable entera cuyo valor se incrementa o decrementa en una cantidad constante (generalmente de 1 en 1) en cada iteración. Se utiliza para registrar la cantidad de veces que ocurre un evento o se repite un ciclo

  Ejemplo:

  ```
  Contador = contador +1;
  ContadorPar = contadorPar+1;
  ```

- **Acumulador:** Es una variable que almacena la suma o multiplicación sucesiva de valores variables en cada iteración. Se utiliza para obtener totales, promedios o sumatorias acumuladas.

  Ejemplo:

  ```
  suma = suma+i;
  suma += i;
  ```

Control de flujo iterativo

C++ proporciona sentencias para alterar la ejecución normal de una estructura repetitiva:

break: Interrumpe y sale inmediatamente del bucle, ignorando las iteraciones restantes

continue: Salta el resto de las instrucciones de la iteración actual y pasa directamente a la siguiente iteración (evaluando de nuevo la condición de incremento).

Escaneado con CamScanner

Estructura repetitiva (while)

Es un bucle o ciclo de control que ejecuta un bucle de código de forma iterativa mientras se cumpla una condición lógica determinada. A diferencia de otros bucles for, el bucle while se utiliza principalmente cuando no se conoce con exactitud el número exacto de iteraciones que se van a realizar antes de iniciar la ejecución.

FUNCIONAMIENTO BÁSICO:

1. Evaluación: Antes de ejecutar el código se evalua la condición lógica
2. Ejecución: Si la condición es verdadera (true) se ejecuta el cuerpo del bucle.
3. Repetición: Al terminar el bloque, se vuelve a evaluar la condición. El proceso continúa hasta la condición pase a ser falsa (false).
4. Finalización: Si la condición es falsa desde el inicio, el bloque dentro del bucle no se ejecutará ni una sola vez.

Estructura repetitiva (do while)

Es un bucle o ciclo de control que ejecuta un bloque de código al menos una vez y luego continua repitiéndolo mientras se cumpla una condición lógica determinada. A diferencia del bucle while tradicional, la condición de evaluación se encuentre al final del código.

FUNCIONAMIENTO BÁSICO

1. Ejecución obligatoria: El programa entra al ciclo y ejecuta las instrucciones del bloque directamente sin comprobar ninguna condición previa.
2. Evaluación posterior: Al llegar al final del bloque, se evalúa condición lógica expresada en el while.
3. Repetición: Si la condición es verdadera (true), el flujo regresa al inicio del bloque para repetir el proceso.
4. Finalización: Si la condición es falsa (false), el ciclo termina y el programa continúa con la instrucción siguiente.

andes

Escaneado con CamScanner

ARREGLOS

Un arreglo (también conocido como array, vector o matriz) es una estructura de datos homogénea y de tamaño fijo que permite almacenar una colección de elementos del mismo tipo bajo un único nombre. Cada elemento individual dentro del arreglo se identifica y se accede mediante uno o más índices numéricos.

Arreglo unidimensional (vector)

Un arreglo unidimensional es una colección lineal de datos que se organizan de forma secuencial, uno al lado del otro en la memoria del computador. Se puede visualizar como una sola fila con múltiples casillas.

Ejemplo:

Indices	0	1	2	3	4
	2	14	15	16	20

Tipo de dato

```
int notas[5];
```

Arreglo bidimensional (Matriz)

Un arreglo bidimensional es una estructura organizada en una tabla de dos dimensiones filas y columnas. Se puede imaginar como una cuadrícula o una hoja de cálculo, donde la intersección de una fila y una columna se determina una posición o casilla específica

Ejemplo:

$$\begin{array}{c} \\ 0 \\ 1 \end{array}\begin{array}{c} \begin{array}{ccc} 0 & 1 & 2 \end{array} \\ \begin{bmatrix} 14 & 15 & 16 \\ 8 & 13 & 20 \end{bmatrix}_{2\times 3} \end{array}$$

```
int matriz[2][3];
```

Escaneado con CamScanner